LT2326 · MACHINE LEARNING · OCTOBER 2026

C.O.De.

Code Origin Detector

Code classification as human-written or AI-written

Andreas Bartsiokas

Can we detect AI-written code?

Tuned code encoders versus n-gram baselines

The difficult case is not AI code written from scratch.
It is AI-edited human code.

Andreas Bartsiokas

Why this matters

Two questions, two levels of difficulty

01 DETECTION

Human or AI?

02 ATTRIBUTION

Which model wrote it?

Core challenge: evaluate on programming problems the model has never seen.

The dataset includes two kinds of AI code

GENERATE

LLM writes a solution from scratch

EDIT

LLM fixes a human submission

7,531

deduplicated samples

1,419

final test files

3

LLM sources

Evaluation designed to avoid easy shortcuts

01	**Group by problem**	No programming problem shared across splits
02	**Normalize code**	Remove whitespace / final-newline tells
03	**Tune on train + dev**	7-fold CV for baseline / dev for encoders
04	**Touch test once**	Fresh seeds / problem-level bootstrap

Three encoder candidates. One strong baseline.

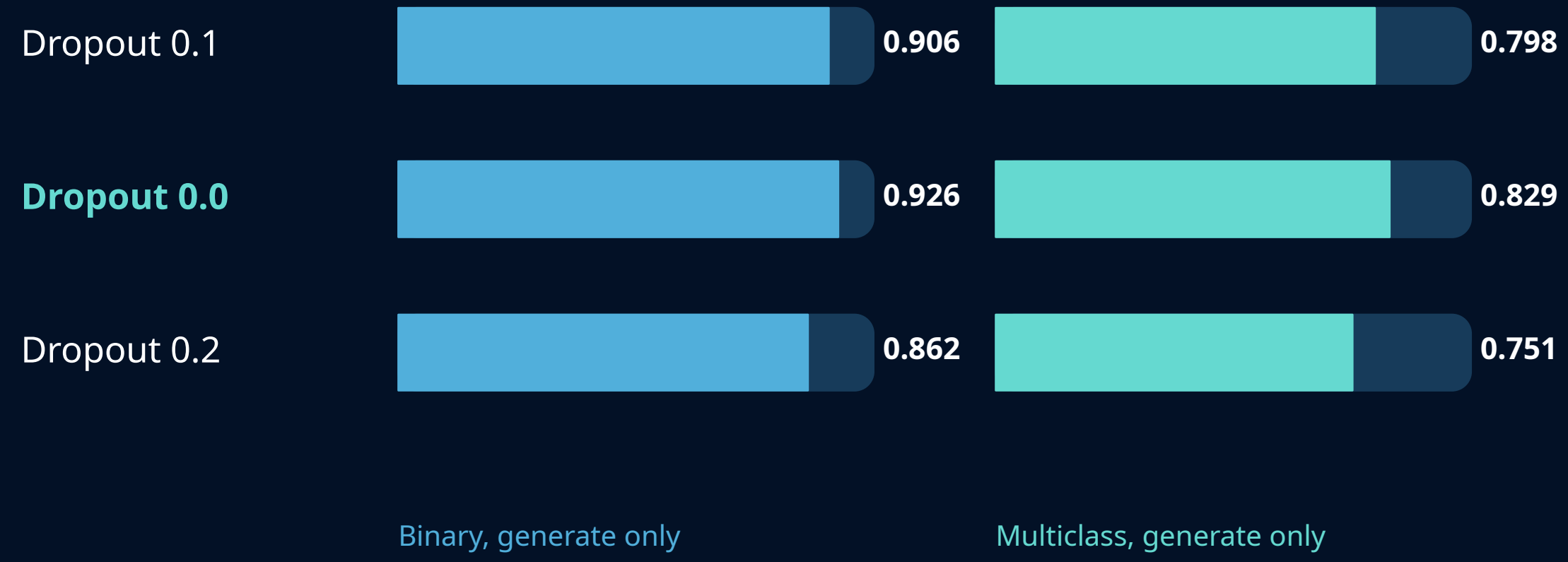
Character n-grams
TF-IDF + logistic regression
CHEAP BASELINE

CodeBERT / GraphCodeBERT
Fine-tuned code encoders
CANDIDATES

UniXcoder
Fine-tuned, 256-token input
FINAL ENCODER

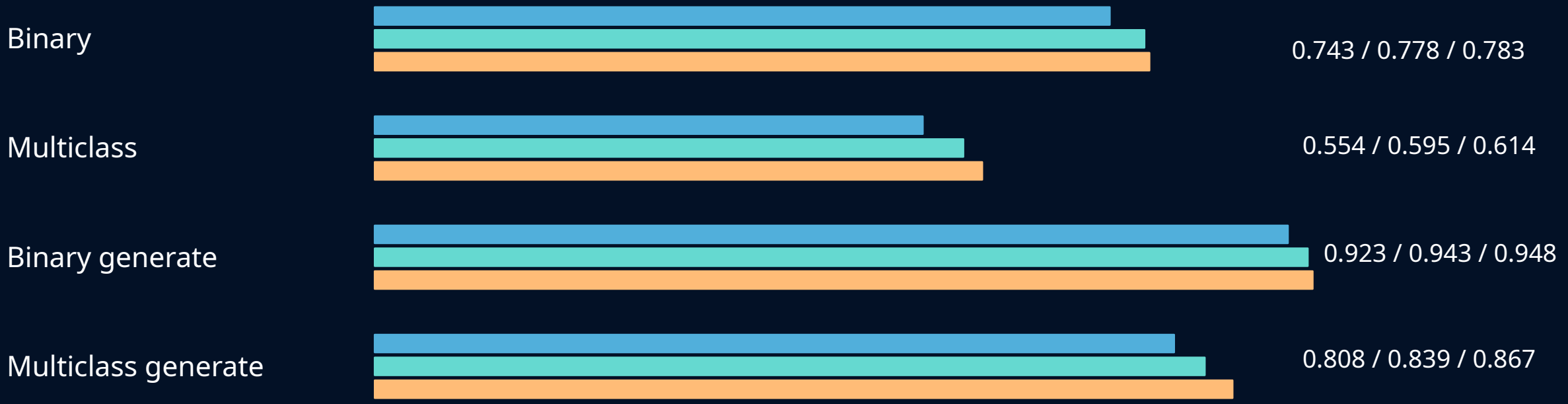
Tuning: removing dropout made the difference

UniXcoder | 256 tokens | development macro-F1



Final results: UniXcoder beats the baseline

Test macro-F1 | all four task variants



CHAR BASELINE

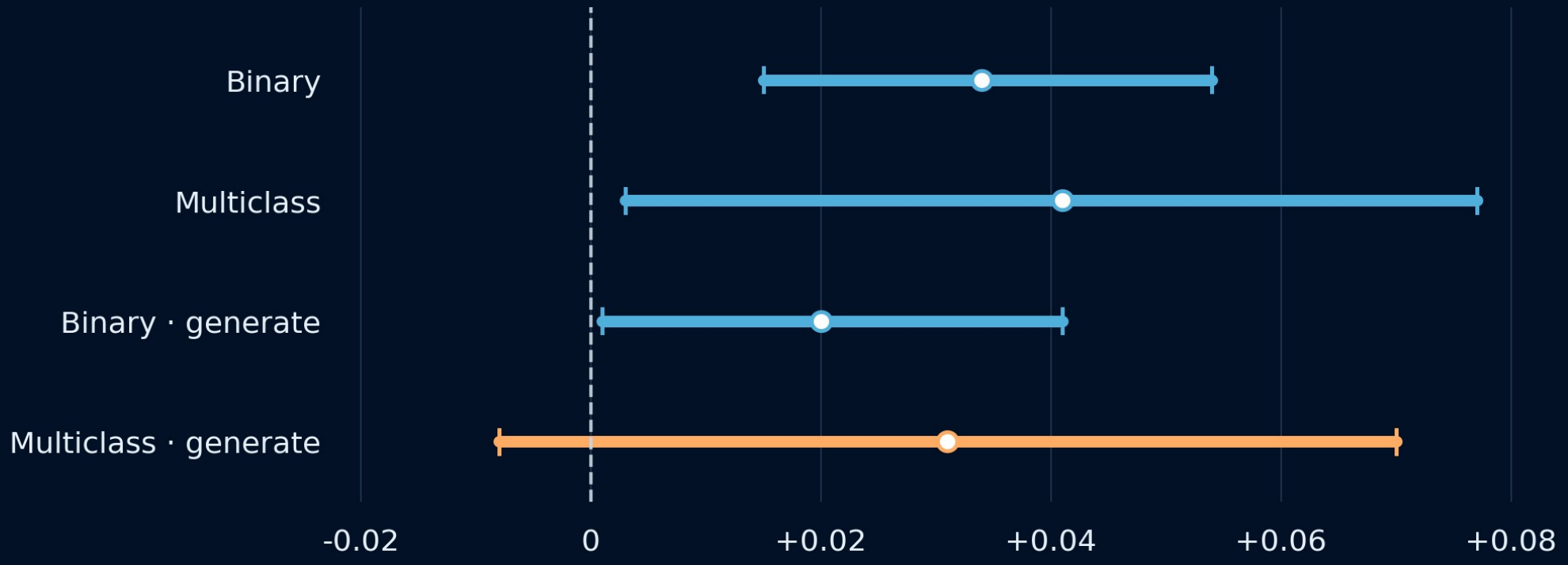
6-SEED ENSEMBLE

50/50 MIX

Values: baseline / ensemble / mix

Are the improvements statistically reliable?

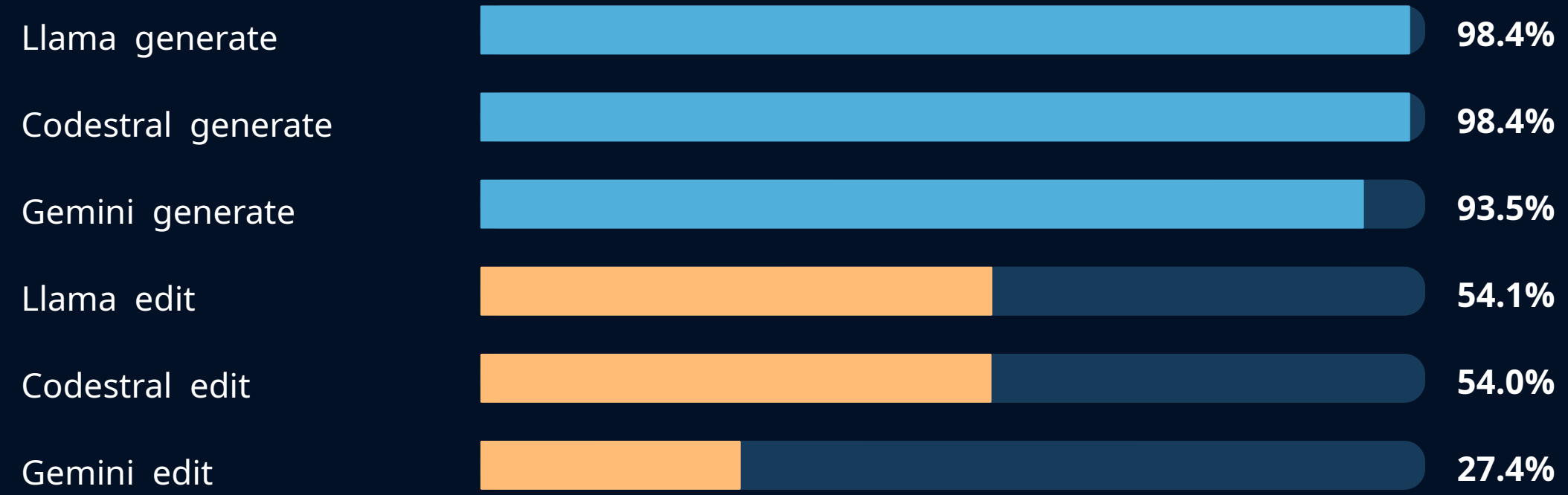
95% problem-level bootstrap intervals => ensemble minus character baseline



Three intervals exclude zero, two only narrowly. Multiclass-generate includes zero.

The central failure: AI-edited code

Share of AI files flagged as AI | binary test | UniXcoder ensemble



328 of 514 AI test files were edits

Attribution: Gemini is mistaken for human

37%

Gemini generate → Human

93%

Llama correctly attributed

79%

Codestral correctly attributed

~10%

human files flagged AI (binary)

What signal is the detector using?

Character baseline ablation | 7-fold training CV | generate-only

	Binary	Multiclass
Full code	0.909	0.791
No layout	0.907	0.778
No comments	0.902	0.787
Identifiers masked	0.882	0.765

What we can and cannot conclude

SUPPORTED

- Character n-grams are strong
- UniXcoder leads on all tasks
- Ensembles help modestly
- Edits are the hardest case

NOT YET TESTED

- Unseen LLMs or new datasets
- Other programming languages
- Why UniXcoder is better
- Why the 50/50 mix helps

Caution: small per-LLM test groups.

Takeaways & next experiments

01 A tuned simple baseline is hard to beat

02 UniXcoder improves macro-F1, especially binary detection

03 Edited code and Gemini expose the real weakness

NEXT → External human-only false-positive test + hold out an LLM

Thank you!! questions?

Can a detector recognize AI assistance, not just AI generation?